

```
Integer i = 4;
int j = i;
Double d = 2;
```

- However, 2 step boxing is not allowed, (line 3: 2 is not automatically casted to a double before auto-boxing)
- Wrapper classes **DO NOT** share the same subtyping relationship as primitives
- Using wrapper classes come at a performance cost, as objects have to be stored on the heap
- Wrapper classes are immutable, changing value involves auto-boxing and auto-unboxing to make a new object

Type Checking

- `a = (C) b`
- Compile Time
 - Find CTT(b)
 - Check if it is possible for RTT(b) <: C, if not Compile Error
- Possibility:
 - CTT(b) <: C: This widening cast is allowed
 - C <: CTT(b): This narrowing cast requires runtime checks for RTT(b)
 - C is an interface: There could be a subclass of B implementing C, so a runtime check is needed
- Impossibility:
 - B and C are unrelated
 - C is an interface and B is final
- Find CTT(a)
- Check if C <: CTT(a), if not Compile Error
- Add run-time check for RTT(b) <: C
- Run Time
 - Find RTT(b)
 - Check if RTT(b) <: C

Variance

- Java Arrays are Covariant: `S <: T` implies `S[] <: T[]`
- Contravariant: `S <: T` implies `T[] <: S[]`
- Invariant: None of the above, not comparable

Exceptions

- Use **try catch finally** keywords
- **throw** immediately suspends execution of the **try** block
- **catch** can catch all exceptions which are subtypes
- **finally ALWAYS RUNS**, unless computer explodes
- Exceptions can be thrown upwards using **throws**
- Unchecked Exceptions: Caused by programming errors, not explicitly caught or thrown, subclasses of run-time errors
- Checked Exceptions: Something programmer cannot control, should be actively anticipated and handled, if not program will not compile

Generics

- Java allows us to define Generic Types that take in Type Parameters
- We can bound type parameters by classes and interfaces using **extends**
- Do not use 2 parameters of the same name in the same context as it confuses the compiler

- Note that generics are **invariant**
- Example Usage

```
class Pair<S, T>{} // Definition
<T> boolean contains(T[] array, T obj){}
A.<String>contains(strArr, str) // Usage
```

Type Erasure

- After type checking, Java performs type erasure to ensure backwards compatibility
- Generic types are replaced by their upper bound, e.g. Comparable (default Object)
- When a Generic is instantiated and used, a typecast is added to the code

```
Int i = new Pair<Str, Int>("a", 4).sec();
Int i = (Int) new Pair("a", 4).sec();
```

Generic Array Problems

- Due to type erasure, we cannot tell if putting Generics into an array will cause errors
- See code below:

```
Pr<Str, Int>[] pArr = new Pr<Str, Int>[2];
Object[] oArr = pArr;
oArr[0] = new Pr<Dbt, Bool>(3.14, true);
```

- becomes

```
Pr pArr = new Pr[2];
Object[] oArr = pArr;
oArr[0] = new Pr(3.14, true);
```

- and this looks okay to us now
- But if we do `Str str = pArr[0].first();` we get a `ClassCastException`
- Arrays are reliable, with full type information available at run-time, unlike generics, so Java does not allow us to make them
- Generic array declaration is ok, but instantiation using **new** is not

Generic Type Rules

- Generic method signature includes type parameters, with them being equal up to renaming
- Type checking uses type argument for class-level type parameters where possible
- Method descriptor stored during dynamic binding at CTT is type erased

Generic Arrays and Warnings

- Since we cannot instantiate generic arrays, we typecast it instead, since arrays are covariant

```
T[] tmp = (T[]) new Object[sz];
this.arr = tmp;
```

- However, this generates the unchecked warning, which can be seen in detail if we compile with `-Xlint:unchecked`
- At compile time, compiler adds typecasts when generics are used

- Unchecked Warning happens when compiler cannot guarantee absence of RTE, due to possible modification of the array
- If we want to suppress it using `@SuppressWarnings("unchecked")`, we need to justify it:
 - array is **private** so it can only be accessed by Seq
 - array can only be modified by `Seq.set(int, T)`, which is of type T
 - The only retrievable objects from array are subtypes of T, using `Seq.get(int)`
 - It is safe to cast `Object[]` to `T[]`
- Another possible warning is raw types, for the same reason where the compiler cannot guarantee type safety due to usage of raw types

Wildcards

- Solves problem with generics being invariant
- Upper-Bounded
 - `copyFrom(Seq<? extends T> src)` allows src to be a subtype of T
 - Is **Covariant**: `S <: T` implies `<? extends S> <: <? extends T>`
 - `<S> <: <? extends S>`
- Lower-Bounded
 - `copyTo(Seq<? extends T> dest)` allows dest to be a supertype of T
 - Is **Contravariant**: `S <: T` implies `<? super T> <: <? super S>`
 - `<S> <: <? super S>`
- PECS: Produce Extends, Consumer Super
 - Think from perspective of the wildcard
 - In `copyFrom`, the wildcard is producing a value
 - In `copyTo`, the wildcard is consuming a value
- Unbounded Wildcards
 - Similar to Object being supertype of all objects, but object as a generic type parameter does not work as generics are invariant
 - Use to replace raw types, does not produce any warnings as no (nonexistent) type information is lost

Type Inference

- We can use an empty diamond operator on RHS to instantiate generic types
- Useful if generic type is really long
- 3 Step Method:
 - Argument Typing: Type of parameters passed in
 - Target Typing: Type of variable where return is stored
 - Type Parameter: Self explanatory
- Java only considers types explicitly involved in the inequalities, then chooses the most specific type
- Example

```
<T extends Circle> T f(Seq<? super T> s)
GetAreable c = f(new Seq<Circle>());
```

- Type Inference
 - Argument Typing: Circle super T, T <: Circle
 - Target Typing: T assigned to GetAreable, T <: GetAreable
 - Type Parameter: T extends Circle, T <: Circle
- Resolved as T <: Circle, so T is Circle

Factory Methods

- Create classes through factory rather than constructor
- Hides constructor information
- Useful when consumer does not know exactly what subtype should be created, acts as auxiliary
- Static method
- Capture generic types to correctly parameterise



pls give me A+ prof thanks